CSE422: Neural Networks Lab
Classwork

Experiment 06: Transfer Learning with Pretrained CNNs

Submission: Completed notebook (.ipynb) with both accuracy outputs printed

Instructions

Follow these steps in order. Do not skip ahead to a later TODO before the earlier ones run without error.

1. **Open a fresh notebook** (Jupyter or Colab) and set the runtime to GPU if using Colab: `Runtime → Change runtime type → GPU`.
2. **Copy the "Setup" block below as-is** into the first cell and run it. It will download CIFAR-10 the first time — this can take a minute; do not interrupt it. Confirm it finished by checking that `train_ds` and `test_ds` print no errors.
3. **Copy the "Part A" block into a second cell.** Before touching any blank, look at the **worked example** at the top of Part A — it shows you exactly what a completed TODO looks like.
4. **Fill in TODOs (1)–(4) one at a time**, in order. For each one: read the comment, find the one or two words it's asking for, and type them in place of the underscores. Don't move to the next TODO until the current one is filled.
5. **Run the Part A cell only after all four blanks in it are filled.** Do not run it with blanks still containing underscores — it will throw a `SyntaxError`.
    - Expect this cell to take a few minutes on GPU (longer on CPU). It is training, not stuck — let it finish.
    - When it finishes, you should see one printed line: `Feature-extraction accuracy: XX.XX%`. Note this number down.
    - If you get an error instead, check the exact TODO comment for the line the traceback points to before asking for help — the fix is almost always in that comment.
6. **Copy the "Part B" block into a third cell** (below Part A — do not overwrite it). This cell continues from the same `model` object Part A already built and trained; you are not starting over.
7. **Fill in TODOs (5)–(7)** the same way — one at a time, checking each against its comment.
8. **Run the Part B cell.** You should now see a second printed line: `Fine-tuning accuracy: XX.XX%`. Note this number down too.

9. **Compare the two numbers** you noted in steps 5 and 8 — you will need this comparison for the submission.
10. **Answer the two written questions** under "What to Submit" in your own words (2–3 sentences each, not copied from the comments).
11. **Save the notebook** (`.ipynb`) with both printed accuracy lines still visible in the cell outputs — do not clear outputs before saving. Submit the file as instructed by your course instructor.

Do **not** change any code outside the blanks — those parts are already correct and are there to show the surrounding context each blank fits into.

Setup (already provided — do not edit)

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models

tf = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

train_ds = datasets.CIFAR10('data', train=True, download=True, transform=tf)
test_ds  = datasets.CIFAR10('data', train=False, download=True, transform=tf)
train_dl = DataLoader(train_ds, 64, shuffle=True, num_workers=2)
test_dl  = DataLoader(test_ds, 128)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Part A — Feature Extraction (frozen backbone)

Worked example — this is the pattern every TODO below follows. If the code were:

```
# TODO (0): Set x to the integer 5.
x = _____________________
```

...the correct completion is simply:

```
x = 5
```

You are always replacing the underscores with a short piece of code — never rewriting the whole line, never adding new lines, unless a TODO explicitly says otherwise (only TODO 7 does).

```
# TODO (1): Load a ResNet-18 pretrained on ImageNet.
#
#       You need: models.resnet18(weights=SOME_VALUE)
#       SOME_VALUE must point to the ImageNet-pretrained checkpoint,
#       not leave the network randomly initialized. Search
#       "torchvision resnet18 pretrained weights" or check the
#       torchvision docs for the exact name of that checkpoint
#       constant (it lives inside models.ResNet18_Weights).
#
#       Check yourself: after this line runs, print(model.fc) should
#       show out_features=1000 (still the original ImageNet head —
#       you replace it in TODO 3).
model = ____________________

# TODO (2): Freeze every parameter so the pretrained weights don't change
#       during training.
#
#       Every PyTorch parameter has a switch called requires_grad
#       (True = will be updated during training, False = frozen).
#       This loop already gives you each parameter, one at a time, as
#       the variable `param`. Set param's requires_grad switch to False.
#
#       Check yourself: param.requires_grad should print False for any
#       parameter after this loop runs.
for param in model.parameters():
    ____________________

# TODO (3): Replace the final layer so the network outputs 10 scores
#       (CIFAR-10 classes) instead of 1000 (ImageNet classes).
#
#       You need: nn.Linear(in_features, out_features)
#       in_features:  use model.fc.in_features (don't type 512
#                     directly — let the code read it automatically)
#       out_features: 10
#
#       Check yourself: print(model.fc) should now show
#       in_features=512, out_features=10.
model.fc = ____________________
model = model.to(device)
```

```
criterion = nn.CrossEntropyLoss()

# TODO (4): Create the optimizer. It should update ONLY the new layer from
#       TODO (3) — not the whole (frozen) network.
#
#       You need: the parameters of just the fc layer, written the
#       same way you write "get all parameters of a model" but on
#       model.fc instead of model.
#
#       Check yourself: this should be a shorter expression than
#       `model.parameters()` — it targets one layer, not the whole
#       network.
optimizer = torch.optim.Adam(_____________________________, lr=1e-3)

for epoch in range(3):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        criterion(model(xb), yb).backward()
        optimizer.step()

model.eval(); correct = 0
with torch.no_grad():
    for xb, yb in test_dl:
        xb, yb = xb.to(device), yb.to(device)
        correct += (model(xb).argmax(1) == yb).sum().item()
print(f'Feature-extraction accuracy: {100*correct/len(test_ds):.2f}%')
```

Part B — Full Fine-Tuning

```
# TODO (5): Unfreeze the entire network — the exact opposite of TODO (2).
#
#       Same loop structure as TODO (2), same `param` variable, just
#       the opposite requires_grad value.
for param in model.parameters():
    _____________________________

# TODO (6): Create a new optimizer that updates the WHOLE network this
#       time (not just fc), using a smaller learning rate than Part A.
#
#       First blank: "all parameters of the model" — you've written
#       this exact expression before, just not assigned to fc this
```

```
#       time.
#       Second blank: a number ten times smaller than 1e-3. Write it
#       the same style (e.g. 1e-3 divided by 10, in scientific
#       notation).
#
#       Why smaller: the backbone weights are no longer frozen, so a
#       large update here would wreck the pretrained features in a
#       few batches (this is called catastrophic forgetting — see
#       Discussion). A small learning rate nudges them gently instead.
optimizer = torch.optim.Adam(_____, lr=_____)

# TODO (7): This blank is different from the others — instead of one or
#       two words, paste in several lines.
#
#       Scroll up to Part A. Select and copy these three pieces, in
#       this order:
#         (a) the `for epoch in range(3):` loop and everything indented
#             under it
#         (b) the `model.eval(); correct = 0` line and the
#             `with torch.no_grad():` block under it
#         (c) the final `print(...)` line
#       Paste all three, in order, in place of the underscores below.
#       Then make ONE change: inside the pasted print statement,
#       change the text 'Feature-extraction accuracy' to
#       'Fine-tuning accuracy' so you can tell the two results apart
#       in your output.
_____
```

What to Submit

1. The completed notebook with both printed accuracy lines visible in the output.
2. One sentence comparing the two accuracies you obtained.
3. Written answers (2–3 sentences each) to:
    - Why did Part A only update `model.fc.parameters()` and not `model.parameters()`?
    - Why is the learning rate in Part B smaller than in Part A?